

MailForge 邮件服务器

SMTP + POP3 + Web + 两层加密 | 课程设计汇报

苏俊玮、王诗贵 | 计算机与网络课程设计 选题 18

2026 年 9 月 11 日

项目简介与课程需求映射

MailForge 是什么

- 原生 Socket 手写 SMTP / POP3 / HTTP。
- 多用户 Web 邮箱：注册、登录、收发、附件、已发送。
- 邮件按 .eml 落盘，重启不丢。
- 两层加密：Web RSA 信封 + 服务器内部/落盘自研 AES/ChaCha。
- 内建协议演示终端与 100 封压测。
- 线程池 + 多账号并发压测；账号密码 PBKDF2-SHA256 随机盐哈希存储。

运行约定

SMTP: 2525 POP3: 1110 HTTP: 8080
默认账号：alice/123456, bob/123456

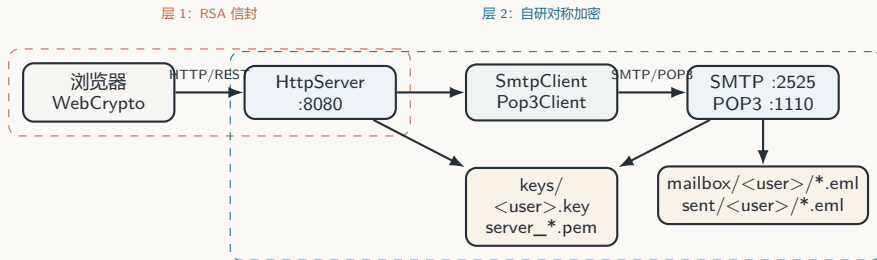
课程要求对照

要求	实现
SMTP 标准交互 POP3 标准交互 正文与附件	EHLO/MAIL/RCPT/DATA/ USER/PASS/STAT/LIST/R MIME multipart + Base64
传输加密 ≥ 2 种算法	两层加密，算法可切换 AES-256-CBC、 ChaCha20
时延/成功率	100 封压测，内建统计

汇报口径

协议正确性用 RFC 状态机和端到端测试证明；
密码算法用 NIST/RFC 官方向量回归证明。

总体架构：三服务器 + 两客户端 + 两加密层



- 三个服务器各占一个线程 accept；每个连接再派发一个工作线程。
- HTTP 后端不直接写文件，而是作为 SMTP/POP3 客户端走本机协议，保证与外部协议客户端一致。

目录结构与模块职责

```
MailForge/
+-- MailServer/
| +-- Server.* 网络基类
| +-- Smtplib.* SMTP 服务端
| +-- Pop3Server.* POP3 服务端
| +-- SmtplibClient.* 内部发信
| +-- Pop3Client.* 内部收信
| +-- HttpServer.* Web 后端
| +-- MailCrypto.* 加密统一入口
| +-- web/ 前端页面
+-- crypto/
| +-- aes.* chacha20.*
| +-- rsa.* random.*
| +-- tests/ docs/
+-- common/
    +-- base64 / logger / file_util / socket
```

分层设计

1. 公共层：Socket、Base64、日志、文件工具。
2. 协议层：SMTP/POP3 服务端与客户端状态机。
3. Web 层：HTTP 解析、REST、会话、MIME、压测。
4. 密码层：MailCrypto 入口；自研 AES/ChaCha + OpenSSL RSA。
5. 展示层：Web 邮箱、协议演示终端。

关键解耦

SMTP/POP3 服务器只处理原文，不感知加密；加密挂在 HTTP 发送前和读取后。

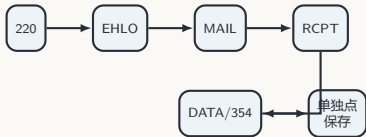
SMTP 原理与标准交互

- SMTP 是基于 TCP 的文本命令/响应协议，服务器问候码 220。
- 核心流程：
 1. EHLO/HELO;
 2. MAIL FROM:<...> 信封发件人;
 3. RCPT TO:<...> 信封收件人;
 4. DATA 开始传正文，单独一行点结束;
 5. QUIT 退出。
- 2xx 成功、3xx 中间态、4xx 暂时失败、5xx 永久失败。
- 本项目实现核心命令，未实现 AUTH、STARTTLS、多收件人队列。

SMTP 对话

```
S: 220 MyMailServer ESMTP ready
C: EHLO client
S: 250 Hello client
C: MAIL FROM:<alice@example.com>
S: 250 OK
C: RCPT TO:<bob@example.com>
S: 250 OK
C: DATA
S: 354 End data with <CR><LF>.<CR><LF>
C: Subject: Hello
C:
C: body
C: .
S: 250 Message accepted
C: QUIT
S: 221 Bye
```

SMTP 状态机、点填充与落盘



- dataMode 区分命令与正文。
- Smtplib 跨命令保存会话。
- 点填充：正文行以点开头写成两个点，服务器还原。

落盘与多用户隔离

1. 从收件人取 @ 前部分，转小写；
2. 存入 mailbox/<user>/时间戳 _ 随机数.eml；
3. 补齐缺失的 Date/From/To/Subject；
4. 头部区 + 空行 + 正文写入文件。

POP3 原理：三阶段状态机与延迟删除

- 服务器先回 +OK，出错回 -ERR。
- 三阶段：
 1. AUTHORIZATION: USER / PASS;
 2. TRANSACTION: STAT / LIST / RETR / DELE / RSET / NOOP;
 3. UPDATE: QUIT 时真正删除被标记邮件。
- DELE 只打标记，中途断开不会误删。
- 多行响应用单独一行点结束，正文点开头行同样点填充。
- 支持新注册账号即时登录：内存表没有则重读 users.txt。

POP3 对话

```
S: +OK MailForge POP3 ready
C: USER bob
S: +OK User bob known
C: PASS 123456
S: +OK mailbox ready, 2 message(s)
C: STAT
S: +OK 2 12345
C: LIST
S: +OK 2 message(s)
S: 1 6000
S: 2 6345
S: .
C: RETR 1
S: +OK 6000 octets
... mail ...
S: .
C: DELE 1
S: +OK message 1 deleted
C: QUIT
S: +OK signing off
```

POP3 实现：快照 + 会话状态

关键结构

- Pop3State: 认证标志、用户名、userKey、邮件快照。
- Pop3Mail: 路径、大小、deleted 标记。
- 登录成功后扫描 mailbox/<user>/*.eml, 排序生成稳定编号。

认证与热加载

normalizeUser() 统一小写并去掉域名;
isUserKnown/checkPassword 自动重读账号文件;
登录成功同时 ensureUserKey()。

- sendMailContent() 逐行读文件, 处理换行与点填充, 不一次性载入大邮件。
- QUIT 时才 unlink() 被标记文件。
- HTTP 后端调用 Pop3Client, 与外部邮件客户端同一套协议。

Server 基类

socket -> bind -> listen -> accept, 每个连接一个线程, 子类实现 `handleClient`。
SMTP/POP3/HTTP 共用这一套骨架。

```
1 class Server {  
2     protected:  
3         virtual void handleClient(int fd) = 0;  
4 };
```

HTTP 后端

- 逐行解析请求行、查询串、头部、Content-Length body。
- POST 表单按 `a=b\&c=d` 解析并 URL 解码。
- 响应固定 Connection: close。
- `/api/login` 用 POP3 试登录, 成功生成 token 存入会话表。

```
1 void HttpServer::handleClient(int fd) {  
2     HttpRequest req; HttpResponse resp;  
3     if (!readRequest(fd, req)) return;  
4     route(req, resp);  
5     sendHttp(fd, resp);  
6 }
```

REST API、MIME 附件与统一.eml 存储

主要接口

路径	功能
/api/register	注册
/api/login	POP3 验证 + token
/api/send	发信; 可选 AES/ChaCha; 可选 RSA 信封
/api/inbox	POP3 LIST + RETR
/api/mail?n=	读信、自动解密、解析附件
/api/attachment	下载附件; web=1 走信封
/api/delete	DELE + QUIT 删除
/api/sent	已发送列表 / 阅读 / 附件
/api/benchmark	100 封 压 测: plain/aes/chacha/all
/api/webkey	下发服务器 RSA 公钥
/api/webpub	上传浏览器 RSA 公钥

MIME 附件

- 生成随机 boundary, 组装 multipart/mixed。
- 附件段使用 Base64。
- 加密时整个 multipart 先整体加密, 再作为正文发送。
- 下载时先解密邮件, 再解析 MIME, 最后 Base64 解码。

统一存储

SMTP 投递、POP3 读取、Web 收发围绕同一份 .eml; 用户目录隔离, 重启不丢。

两层加密模型：信任边界不同



层 1：浏览器到服务器

- 本机无 TLS 时保护 Web 表单、收件箱、附件。
- AES-256-GCM 加密内容，RSA-OAEP-2048 包装会话密钥。
- 浏览器 WebCrypto，服务器 OpenSSL EVP。

层 2：服务器内部与磁盘

- 邮件正文和附件整体加密后再走 SMTP/POP3/落盘。
- AES-256-CBC 或 ChaCha20 可切换。
- 纯 C++ 自研，不依赖 OpenSSL。
- 发送副本用收件人密钥，已发送副本用发件人密钥。

层 1: RSA 数字信封与 WebCrypto 互通

发送: 浏览器到服务器

1. 浏览器生成一次性 AES-256-GCM 会话密钥;
2. 用服务器 RSA 公钥做 RSA-OAEP(SHA-256) 封装;
3. 表单用 AES-GCM 加密, 末尾带 16B tag;
4. 服务器用私钥拆封, 再解 GCM。

反向: 服务器到浏览器

浏览器生成 RSA-2048 密钥对, 公钥上传 /api/webpub; 服务器读信、附件、压测结果用该公钥封装返回。

```
k=<Base64 RSA-OAEP-SHA256(32B session key)>
|iv=<Base64 12B GCM IV>
|ct=<Base64 AES-GCM(ciphertext || 16B tag)>
```

- WebCrypto 导入 SPKI 公钥; JWK 私钥存 localStorage。
- 服务器 OpenSSL 读写 PEM, 双方标准算法互通。
- WebCrypto 不可用时前端降级明文 HTTP。

层 2: 自研 AES-256-CBC 与 ChaCha20

AES-256-CBC

- 分组密码: 块 16B, 密钥 32B, 14 轮。
- CBC 链接, PKCS#7 填充并校验。
- 每封邮件随机 16B IV。
- 线格式: `MailForge::ENC::AES::` + `Base64(IV || 密文)`。

ChaCha20

- 流密码: 密钥 32B, nonce 12B, 64B 密钥流块。
- ARX: 加法、异或、循环移位, 无查表。
- 20 轮; 每封邮件随机 nonce。
- 线格式: `MailForge::ENC::CHA::` + `Base64(nonce || 密文)`。

工程约定与安全边界

真实 Subject + 空行 + 正文/附件 multipart 整体加密; 头部只留占位 Subject 与 X-MailForge-Crypto。AES-CBC 和 ChaCha20 本身都不提供完整性认证, 建议升级为 AES-GCM 或 ChaCha20-Poly1305。

AES-256 原理与实现要点

四个变换

1. SubBytes: S 盒非线性替换;
2. ShiftRows: 按行循环左移;
3. MixColumns: $GF(2^8)$ 上乘固定矩阵;
4. AddRoundKey: 与轮密钥异或。

AES-256 密钥扩展为 15 个轮密钥; 最后一轮不做 MixColumns。

S 盒不是抄表

在 $GF(2^8)$ 中求逆 $a^{-1} = a^{254}$, 再做仿射变换; 逆 S 盒由正向映射反推。

```
1 bool Aes::Encrypt(key, iv, plaintext, ciphertext) {
2     size_t pad = 16 - (plaintext.size() % 16);
3     // PKCS#7 填充
4     ExpandKey(key.data(), rk.data());
5     for each block:
6         blk = plaintext_block ^ prev;
7         EncryptBlock(rk, blk, out);
8         prev = out;
9 }
```

正确性验证

使用 NIST SP 800-38A F.2.3 官方向量回归, 逐块比对 AES-256-CBC 输出。

ChaCha20 原理与实现要点

状态与轮函数

16 个 32 位字：4 常量 + 8 密钥字 + 1 计数器 + 3 nonce 字。

每轮做列轮与对角轮的 QuarterRound：

$a+ = b$; $d \oplus = a$; $d \lll 16$; $c+ = d$; $b \oplus = c$; $b \lll 12$

共 10 次双轮，最后累加初始状态并输出 64B 密钥流。

验证

使用 RFC 8439 §2.3.2 官方向量验证 Block()。

```
1 void ChaCha20::Block(key, nonce, counter, out) {
2     uint32_t state[16], work[16];
3     state[0..3] = constants;
4     state[4..11] = key words;
5     state[12] = counter;
6     state[13..15] = nonce words;
7     for (int round = 0; round < 10; ++round) {
8         // column + diagonal QuarterRound
9     }
10    for (int i = 0; i < 16; ++i)
11        Store32LE(work[i] + state[i], out + 4*i);
12 }
```

密钥管理与加解密挂钩点

密钥文件

- 每账号 `keys/<user>.key`: 32B 随机密钥;
- 首次使用自动生成;
- 用户名规范化: 取 @ 前、转小写;
- 全局互斥锁保护查文件加写文件。

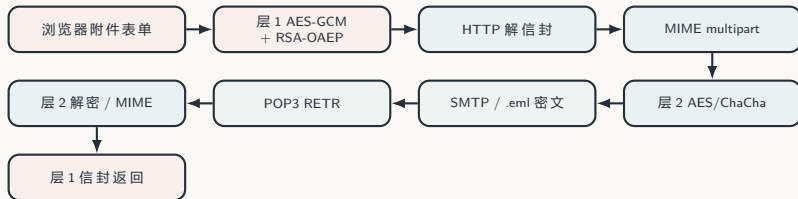
服务器 RSA 密钥

`keys/server_public.pem` /
`server_private.pem`, 首次启动自动生成。

加解密挂钩点

1. `handleSend()`: 用收件人密钥加密载荷;
2. `decodeMail()`: 按魔数识别算法, 用查看者密钥解密;
3. 已发送副本: 用发件人密钥再加密一份;
4. 附件下载: 先解密邮件, 再解析 MIME, 再 Base64 解码。

端到端数据流与测试结果



正确性测试

- AES-256-CBC: NIST SP 800-38A F.2.3.
- ChaCha20: RFC 8439 §2.3.2.
- RSA 信封: Seal/Open 双向、篡改和错密钥拒绝。
- 协议端到端: SMTP 发信 -> 落盘 -> POP3 收信。

性能压测

每模式 100 封约 1MB; 支持
threads=1/2/4/8/16 与多账号轮询。实测单线程约 15.7 封/s, 4 线程约 54.3 封/s, 丢包率 0%。

注意

以上为本地环回演示数据; 真实网络评测应补充公网、并发、P95/P99 等指标。

创新点总结

密码学创新

1. 两层加密模型：Web 链路与服务器内部/落盘分别保护。
2. 纯 C++ 自研 AES-256-CBC 与 ChaCha20，过 NIST/RFC 官方向量。
3. WebCrypto 与 OpenSSL 互通，标准线格式。
4. 加密与协议解耦，SMTP/POP3 对密文透明。

系统工程创新

5. 协议、Web、存储统一 .eml，不绕开协议。
6. 浏览器协议演示终端 + 100 封压测 + 自动清理。
7. 零配置密钥：首次使用自动生成。
8. 已发送副本用发件人密钥再加密，双方都能读。

一句话

既展示了协议状态机，又展示了密码算法内部结构，并且都有可运行、可验证的闭环。

总结与答辩准备

总结

- 从零手写 SMTP、POP3、HTTP。
- 多用户 Web 邮箱、MIME 附件、.eml 持久化。
- 两层加密：RSA 信封 + 自研 AES/ChaCha。
- 官方向量、端到端、100 封压测验证。

核心收获

协议要处理粘包/半包、点填充；密码算法要用官方向量证明；加密要按信任边界分层；安全加固与工程化同样重要。

高频问答

- 粘包/半包：缓冲区按换行拆行，半行留到下次。
- POP3 删除：DELE 只标记，QUIT 才真正删除。
- 为什么两层：浏览器链路和服务端内部/磁盘信任边界不同。
- 自研算法安全吗：教学验证用，生产应用审计过的 AEAD。

谢谢

MailForge：让邮件协议和安全算法都看得见、跑得通、验得过。